

Sesión 1 de Ako, contada fácil

Resumen para leer tranquilo, sin prisa. Todo lo que hicimos el 28 y el 29 de septiembre, explicado como si montáramos un bar.

Primero, el mapa: ¿cómo se hace una app de Android?

Imagina que quieres abrir un bar. Necesitas:

- **Un local donde montarlo** → **Android Studio**. Es el programa donde escribes la app, igual que Word es el programa donde escribes un documento.
- **Los materiales y herramientas de construcción** → **el SDK de Android**. Es una caja enorme con todo lo que Google da para hacer apps: piezas de pantalla, botones, cómo se habla con el móvil... Lo tenemos en `C:\Android\Sdk`.
- **Un local de pruebas antes de abrir al público** → **el emulador**. Es un móvil de mentira dentro de tu ordenador. Así pruebas la app sin tener que pasarla a un móvil de verdad.
- **Un encargado que junta todo y deja el bar listo para abrir** → **Gradle**. Coge tu código, las piezas del SDK y las librerías, y lo empaqueta en una **APK**, que es el archivo que se instala en el móvil.
- **Un libro de caja donde apuntas cada cambio** → **Git**.
- **Una copia de ese libro en la nube, a la vista de todos** → **GitHub**.

Con eso ya tienes el dibujo completo. Todo lo de esta sesión fue montar esas seis cosas.

Lo que hicimos, paso a paso

1. Abrimos Android Studio por primera vez

Al arrancar, Android Studio quería guardar el SDK en una carpeta escondida (`AppData`). Le dijimos que no: el SDK ya estaba en `C:\Android\Sdk`. Es como cuando el de la mudanza quiere dejar las cajas en el trastero y tú le dices «no, que ya las tengo colocadas en la cocina».

2. Creamos el proyecto

Elegiste la plantilla **Empty Views Activity**, que es una app vacía con una sola pantalla que dice «Hello World!». Rellenaste:

- **Nombre:** AKO. Es lo que sale debajo del icono en el móvil.
- **Package name:** `yunkang.ako`. Es **el DNI de la app**: Android distingue las apps por este nombre, y dos apps no pueden tener el mismo. Por eso no se cambia nunca. También sirve para ordenar tu código en carpetas (`yunkang/ako/`).

- **Minimum SDK: API 26 (Android 8).** Es la **edad mínima para entrar al bar**: la app funciona en móviles con Android 8 o más nuevo. ¿Por qué 8? Porque el PIN del Propietario se guarda «picado» con una receta de seguridad llamada **PBKDF2 con SHA-256**, y esa receta viene de serie en Android a partir del 8. Con una versión más vieja habría que traer una librería de fuera, y el diseño no lo permite. Además, el 98,4 % de los móviles ya tienen Android 8 o más.

3. La app arrancó en el emulador

Le diste al botón verde **Run ▶**, se encendió el móvil de mentira y salió «Hello World!». Primer entregable hecho: **la app vacía arranca**.

4. Pusimos las librerías del proyecto

Las **librerías** son **herramientas que ya ha fabricado otra gente** y que usas en vez de hacerlas tú. Por ejemplo, no construyes tú la nevera del bar: la compras. En Ako:

- **Room** es **la nevera**: donde se guardan los datos (platos, mesas, comandas).
- **Glide** es **el álbum de fotos**: enseña las fotos de los platos.
- **Las corrutinas** son **un camarero que hace los recados largos** mientras tú sigues atendiendo, para que la pantalla no se congele.
- **ViewBinding** es **poner etiquetas a los botones de la barra**: en vez de buscarlos a ciegas, los llamas por su nombre.
- **KSP** es **un pinche**: tú escribes «quiero una nevera así» y él te monta los estantes solo.

¿Dónde se apunta qué librerías usamos y de qué versión? En **el catálogo de versiones**, un archivo llamado `libs.versions.toml`. Es **la lista de la compra con las marcas**: «Room, versión 2.8.5; Glide, versión 5.0.9...». Ojo, que esto costó: **no son las versiones de tu proyecto**, son las versiones de las herramientas que usa tu proyecto. Si mañana quieres otra versión de Room, cambias una línea de esa lista y ya.

Estos archivos los copié yo, porque son configuración y no el código de la app. Tú los sincronizaste en Android Studio (**Sync Now**) y la app volvió a arrancar. Apareció una carpeta nueva, `java (generated)`, con código que fabricó el pinche (KSP) solo: la prueba de que todo funcionaba.

5. Git y GitHub

- **Un commit** es **una foto del código en ese momento**, con una nota de qué cambió. Si algo se rompe mañana, puedes volver a esa foto. (Lo dijiste bien: «es como está el código en ese momento».)
- **Un push** es **mandar esas fotos a GitHub**. (También bien: «envías el código a un sitio».)
- **GitHub** es la copia en la nube. El repositorio `ako` es **público**: lo puede ver tu profesor, y si se rompe tu ordenador no pierdes nada.

6. El README

Es **la portada del repositorio**: lo primero que ve quien entra en GitHub. Explica qué es la app, cómo abrirla, cómo probarla y cómo se usó la IA. Casi lo tenías: no es algo de la app en el móvil, sino **del repositorio**.

7. La consulta al profesor

Te preocupaba que usar IA diera problemas, así que se lo preguntaste. Te dijo que **sí, que está permitido**: la IA es una herramienta más. Lo malo sería que la IA hiciera el proyecto sola o que dependieras de ella. Desde entonces:

- Los commits llevan la línea **«Co-Authored-By: Claude»**: tú eres el autor y Claude sale como ayudante.
- **Tú eliges las decisiones de código**: yo te doy al menos **3 opciones**, una mía y otra sacada de internet con su enlace, y tú decides. Como pedir presupuesto a tres fontaneros.
- **Somos un equipo**: si algo se tuerce, paro, te lo cuento y lo resolvemos juntos.

8. Seguridad

Quitamos tus correos de los documentos públicos y revisamos todo el historial buscando claves de API: **limpio**. Además, el hook (un **portero** que revisa cada commit antes de apuntarlo) ahora **mira el DNI**: si el commit no va a tu nombre, no lo deja pasar. Ese bloque lo pegaste tú.

Lo que repasamos al empezar la S2

Me dijiste que en general te ha costado entenderlo todo porque nunca habías usado Android Studio. Es normal. Estas tres cosas las repasamos antes de escribir código:

1. **Gradle**: el encargado que junta todo y fabrica la APK.
2. **El catálogo de versiones**: la lista de la compra con las marcas de las herramientas (no las versiones de tu proyecto).
3. **Por qué minSdk 26**: por la receta de seguridad del PIN, que viene de serie desde Android 8.

Una frase para el vídeo

«Creé el proyecto en Android Studio con `minSdk 26` porque el hash del PIN (PBKDF2 con SHA-256) viene de serie desde esa versión; Gradle compila la app y las versiones de todas las librerías están en un solo sitio, el catálogo `libs.versions.toml`; el código está en un repositorio público de GitHub desde el primer día.»